

Comparing Supervised Models and the Impact of PCA Dimensionality Reduction (Wine Dataset)

Student id: 10263704
Name: Subathra Sundarbabu
Course Code: CM3015
Date: 7th January 2026

Table of Contents

1 Abstract	2
2 Introduction	2
2.1 Machine Learning Context	2
2.2 Aim	3
2.3 Dataset Description	3
2.4 Dataset Limitations	4
2.4.1 Short Sample Size	4
2.4.2 Correlation and Dimensionality Issues	4
2.4.3 Model Performance and Evaluation	4
3 Background	4
3.1 K-Nearest Neighbours (KNN)	5
3.2 Decision Tree Classifier	5
3.3 Gaussian Naïve Bayes	6
3.4 Principal Component Analysis (PCA)	6
3.5 Model Evaluation and Validation Tools	7
4 Methodology	7
4.1 Data Exploration	7
4.2 Visualisations	8
4.2.2 Class Distribution	8
4.2.3 Feature Distributions	9
4.2.4 Correlation Heatmap	10
4.3 Dataset Splitting and Feature Scaling	11
4.4 K-Nearest Neighbours (Build from scratch)	12
4.5 Other Model Training	14
4.6 Evaluation Metrics and Cross-Validation	14
4.7 Dimensionality Reduction using PCA	15
5 Results	15
5.1 Train-Test split Performance	15
5.2 Confusion Matrix and Learning Curves	16
5.2.1 KNN	16
5.2.2 Decision Tree	17
5.2.3 Gaussian Naive Bayes	19
5.3 Stratified 5-Fold Cross-Validation Results	20
5.4 Dimensionality Reduction using PCA results	20
6 Evaluation	21
6.1 Model Performance Relative to Aim	21
6.2 Advantage of the Approach	21
6.3 Impact of the Limitation of dataset	22
6.4 Evaluation of algorithms	22
6.5 Methodological Constrains and Future Improvements	22
7 Conclusions	23
8 References	23

1 Abstract

To compare model performance, understand algorithmic behaviours, and evaluate the effects of feature scaling, dimensionality reduction, and validation techniques on classification accuracy, this project explores the application of supervised machine learning techniques to the classical Wine dataset. Three different classification models were used by the chosen algorithms: an instance-based statistical approach (KNN), a probabilistic generative baseline (Gaussian Naive Bayes), and a rule-based learner (Decision Tree). This combination makes it possible to compare clearly how various biases impact generalisation actions, response to scaling, and decision boundaries on the same dataset. KNN was implemented from scratch to improve comprehension of distance metrics, neighbouring voting, and the practical impact of preprocessing decisions. For fair evaluation, a structured method was used, starting with exploratory data analysis to look at feature distributions, class balance, and inter-feature correlations. Data standardisation and an 80-20 train test split come next. Accuracy, confusion matrices, and classification reports were used to evaluate the model's performance, enabling class-level examination of precision, recall, and F1-score. K-fold cross-validation was used to compare classifiers under repeated resampling to address concerns about overfitting. Principal Component Analysis (PCA) was also used to investigate how dimensionality reduction affected model resilience and classification performance. The findings show that probabilistic and distance-based models perform well on this well-structured dataset, with the Decision Tree model showing greater sensitivity to data variation and KNN and Gaussian Naive Bayes reaching high classification accuracy. PCA highlighted the redundancy in the original feature space while reducing dimensionality with little decrease in performance. Overall, the results demonstrate that meticulous preprocessing, validation, and model selection are essential for reliable machine learning results, and that simpler models can compete when the dataset is well prepared and performs well.

2 Introduction

The project is described in this section in relation to supervised machine learning research. It explains the significance of algorithm comparison for wine classification, makes the project's goal clear, and presents the dataset and issues found in earlier research.

2.1 Machine Learning Context

For classification problems in fields where results depend on several numerical features, supervised machine learning approaches are often used. To find patterns and predict class, algorithms like K-Nearest Neighbours (KNN), Decision Trees, Naive Bayes classifiers, and neural-network-based models are widely used on structured datasets. Previous research has shown that wine quality and wine type classification offer a useful platform to evaluate these algorithms because wine datasets have clearly labelled classes and well-defined chemical attributes (Pradhana, Batubulan, Kotama, 2024; Khilari et al., 2022).

The Wine dataset is also a well-established and frequently used benchmark. Its small size and tidy, well-separated structure can result in positive performance estimates that might not apply to more complex, real-world classification tasks. As a result,

rather than being a claim of general external validity, the results on Wine should be seen mainly as evidence about algorithmic action under conditions.

2.2 Aim

This project's aim is to investigate compare various supervised classification methods on a wine dataset while looking at how preprocessing, model assumptions, and validation techniques affect classification results. The dataset was chosen not only for simple of use but also because it offers a controlled, repeatable environment in which variations in model assumptions can be separated and looked at. This helps achieve the learning goal of the project, which is to compare validation stability, interpretability, and preprocessing between different models.

Because model selection decisions in real-world machine learning applications sometimes involve choices between accuracy, interpretability, computing cost, and robustness rather than simply optimising predictive performance, this goal is both interesting and relevant. This effort focus algorithmic understanding by building a K-Nearest Neighbours classifier from scratch and comparing it against well-known library models, in contrast to studies that just concentrate on attaining the maximum possible accurac. More analysis of how various algorithms react to data properties like feature scaling, correlation, and dimensionality is seen in this project. This viewpoint corresponds with recommendations found in the literature that highlight the significance of understanding model behaviours, interpretability, and generalisation performance in addition to basic evaluation metrics (Jiang, 2024).

2.3 Dataset Description

The scikit-learn library's Wine dataset, which came from a chemical study of wines made from three different grape cultivars in the same area of Italy, was used in this project. Alcohol level, acidity, colour intensity, flavonoids, magnesium, and phenolic compounds are among the 13 continuous chemical characteristics that define each sample. There is a three-class classification challenge since the target variable is a category label that identifies the wine cultivar. Wine allows transparen comparison with earlier research and improves the reproducibility of experimental results. Therefore, the dataset in this project is academic, it is used to assess modelling choices and validation behaviours.

												od280/od315_of_diluted_wines	proline	target	
	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue				
0	14.23	1.71	2.43		15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065.0	0
1	13.20	1.78	2.14		11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050.0	0
2	13.16	2.36	2.67		18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185.0	0
3	14.37	1.95	2.50		16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480.0	0
4	13.24	2.59	2.87		21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735.0	0

Img 1: First 5 rows of the dataset

Because of their numerical structure, moderate dimensionality, and clearly defined class boundaries, wine datasets of this kind are often used in machine learning research (Pradhana, Batubulan and Kotama, 2024). For easier statistical examination and exploratory data analysis, the dataset in this project is loaded and transformed into a Pandas Data Frame. Distance-based classifiers have difficulties

because to the large range in feature scales, even though the dataset has no missing values and requires little cleaning. This feature encourages the use of feature standardisation before model training, which is frequently said in the literature for algorithms like neural networks and KNN (Jiang, 2024).

2.4 Dataset Limitations

2.4.1 Short Sample Size

The Wine dataset has several drawbacks that affect how models are evaluated and interpreted, despite its widespread use. One significant drawback is the short sample size, which makes flexible models like Decision Trees more vulnerable to data partitioning and increases the danger of overfitting. Previous research has shown that even slight changes in the training data can result in distinctly different decision boundaries and tree architectures (Khilari et al., 2022).

2.4.2 Correlation and Dimensionality Issues

The dataset's inclusion of associated attributes presents another difficulty. If strong correlations between chemical characteristics are not appropriately addressed through scaling or dimensionality reduction, they might skew distance estimates and lower the efficiency of some classifiers. Principal Component Analysis (PCA), which reduces redundancy while maintaining most of the dataset's variation, is used in several research to address this problem (Jiang, 2024).

2.4.3 Model Performance and Evaluation

Research on wine classification that has already been done often reports good accuracy for a variety of models, such as KNN, neural networks, and ensemble methods (Khilari et al., 2022). These findings show that the dataset is separable, but they also raise question on whether advanced models outperform simpler ones in terms of performance. As a result, in addition to classification accuracy, this project also highlights comparative analysis, validation stability, and interpretability, which aligns the experimental design with methodological and educational objectives stated in the academic literature.

2.4.4 Popular and Easy dataset

Another drawback is that Wine is a popular and easy dataset many models achieve high accuracy, which may reduce the significance of differences in overall performance. Therefore, rather than claiming broad real-world generalisation from a single benchmark, the focus of this report is on comparative behaviours sensitivity to scaling, stability under cross-validation, and performance under dimensionality reduction.

3 Background

In supervised classification, a set of input features and a known categorical target variable are mapped using labelled training data. Building a model that can generalise this mapping to unseen data is the goal. Based on chemical measures, each wine sample in this

experiment is classified into one of three cultivar classes using a multiclass classification system. Distance-based learn (KNN), Gaussian Naive Bayes, and Decision Trees are the three modelling approaches chosen for this project. This diversity makes it possible to compare how various assumptions and biases impact the behaviour and performance of models in a meaningful way.

3.1 K-Nearest Neighbours (KNN)

A distance-based, non-parametric classification technique called K-Nearest Neighbours gives a test sample a class label based on the majority class of its k closest training samples in feature space. KNN lacks a clear decision boundary during training, in contrast to structural models. Rather, classification waits until prediction time, and all training data are kept.

This project shows an understanding of the workings of KNN by implementing it from scratch in Python. The Euclidean distance metric is used by the algorithm to calculate pairwise distances between a test instance and each training sample. The most common class label among those neighbours is given as the prediction after the k shortest distances are chosen. This implementation highlights the significance of neighbourhood selection and distance calculations, which are important to display KNN behaviour.

KNN is extremely sensitive to feature scale since it depends directly on distances. Distance estimates may be disproportionately influenced by features with wider numerical ranges, which could result in biased classifications. Because of this, standardisation of features using `StandardScaler` was implemented before training and evaluation. By ensuring that every feature contributes equally to the distance metric, this preprocessing stage improves prediction stability and fairness.

3.2 Decision Tree Classifier

Supervised learning models called decision trees divide the feature space according to decision rules to classify data. The algorithm chooses a feature and threshold at each internal node based on an impurity metric like information gain that best separates the data. Impurity measures commonly used are Gini impurity and entropy (Abhinav, 2025):

$$Gini = 1 - \sum_{i=1}^n p_i^2$$

$$Entropy = - \sum_{i=1}^n p_i \log_2(p_i)$$

The final class predictions are shown by leaf nodes. This project made use of the scikit-learn library to create the Decision Tree classifier. Decision rules are visualised and easily understood, which is one of Decision Trees main advantages. This model also shows significant variation, even small changes to the training data can produce distinct tree architectures. The project results show this sensitivity, as Decision Tree performance differed more than other models.

Decision Trees, in contrast to KNN, do not need feature scaling because they use threshold-based splits instead of distance calculations. However, validation techniques like cross-validation are important in assessing their ability to generalise performance because of their tendency to overfit small datasets.

3.3 Gaussian Naïve Bayes

Gaussian Naive Bayes is a probabilistic classifier that is predicated on the idea that characteristics are conditionally independent given the class label and on Bayes' theorem. Bayes Theorem (Neha, 2025):

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

The Gaussian variant assumes that feature values for continuous features are distributed normally throughout each class. Scikit-learn was used in this project to implement the Gaussian Naive Bayes model. Each feature's class-specific means and variances are estimated by the model during training. It determines the class with the highest probability during prediction by calculating each class posterior probability based on the observed feature values.

Gaussian Naive Bayes often performs well in practice because of its simplicity and robustness, even if the independence assumption is broken in real-world datasets. Strict rules to assumptions are not necessarily required for successful prediction, as shown by the model's good classification performance in the Wine dataset despite the existence of correlated characteristics.

3.4 Principal Component Analysis (PCA)

An unsupervised dimensionality reduction method called Principal Component Analysis reduces the original feature space to a smaller collection of components that best represent the data's highest variance. PCA projects the data onto the directions that show the greatest variation in the data.

After feature normalisation, PCA was used in this project to see how dimensionality reduction affected classification performance. The explained variance ratio was used to determine the amount of information contained by each basic component. Then, models were re-trained on the reduced feature space to see if performance could be maintained with fewer dimensions.

The two goals of using PCA are to increase computational efficiency and decrease redundancy brought on by correlated features. Additionally, it provides light on the dataset's underlying structure. The findings showed that classification accuracy could be maintained even after dimensionality reduction, suggesting that overlapping information was present in a large portion of the initial feature space.

3.5 Model Evaluation and Validation Tools

Several assessment measures were used to ensure a fair and reliable comparison of models. Insights into precision, recall, and F1-score were obtained using confusion matrices and classification reports in addition to the primary indicator of classification accuracy. These metrics show the distribution of errors among classes as well as the frequency of model accuracy.

Furthermore, k-fold cross-validation was used to reduce the dependency on a single train-test split. Cross-validation shows how sensitive a model is to data variance and offers a more reliable estimate of generalisation performance by repeatedly training and testing models on various data sections. This validation approach was important because of the dataset's small size.

4 Methodology

This section explains the experimental methodology that was used to prepare the data, investigate the dataset, apply machine learning models, and assess their effectiveness. Data preparation, model implementation, validation, dimensionality reduction, and exploratory data analysis are the first steps in the process.

4.1 Data Exploration

To evaluate the Wine dataset's feature characteristics, and suitability for supervised classification, an initial look at was carried out. For every feature, summary statistics were produced that included measurements of dispersion and central tendency. characteristics like proline and magnesium showed greater numerical scales than characteristics like ash and colour, according to these statistics, which also showed significant diversity in feature ranges. This finding supported the necessity of defining features before using distance-based methods.

	count	mean	std	min	25%	50%	75%	max
alcohol	178.0	13.000618	0.811827	11.03	12.3625	13.050	13.6775	14.83
malic acid	178.0	2.336348	1.117146	0.74	1.6025	1.865	3.0825	5.80
ash	178.0	2.366517	0.274344	1.36	2.2100	2.360	2.5575	3.23
alcalinity of ash	178.0	19.494944	3.339564	10.60	17.2000	19.500	21.5000	30.00
magnesium	178.0	99.741573	14.282484	70.00	88.0000	98.000	107.0000	162.00
total phenols	178.0	2.295112	0.625851	0.98	1.7425	2.355	2.8000	3.88
flavanoids	178.0	2.029270	0.998859	0.34	1.2050	2.135	2.8750	5.08
nonflavanoid phenols	178.0	0.361854	0.124453	0.13	0.2700	0.340	0.4375	0.66
proanthocyanins	178.0	1.590899	0.572359	0.41	1.2500	1.555	1.9500	3.58
color intensity	178.0	5.058090	2.318286	1.28	3.2200	4.690	6.2000	13.00
hue	178.0	0.957449	0.228572	0.48	0.7825	0.965	1.1200	1.71
od280/od315 of diluted wines	178.0	2.611685	0.709990	1.27	1.9375	2.780	3.1700	4.00
proline	178.0	746.893258	314.907474	278.00	500.5000	673.500	985.0000	1680.00
target	178.0	0.938202	0.775035	0.00	0.0000	1.000	2.0000	2.00

Img 2: Range Checks

```
alcohol      0      alcohol      float64
malic_acid   0      malic_acid     float64
ash          0      ash          float64
alcalinity_of_ash  0      alcalinity_of_ash  float64
magnesium    0      magnesium    float64
total_phenols 0      total_phenols  float64
flavanoids   0      flavanoids    float64
nonflavanoid_phenols 0      nonflavanoid_phenols float64
proanthocyanins 0      proanthocyanins float64
color_intensity 0      color_intensity float64
hue          0      hue          float64
od280/od315_of_diluted_wines 0      od280/od315_of_diluted_wines float64
proline      0      proline      float64
target       0      target       int32
dtype: int64      dtype: object
```

Img 4: Null and data types checking

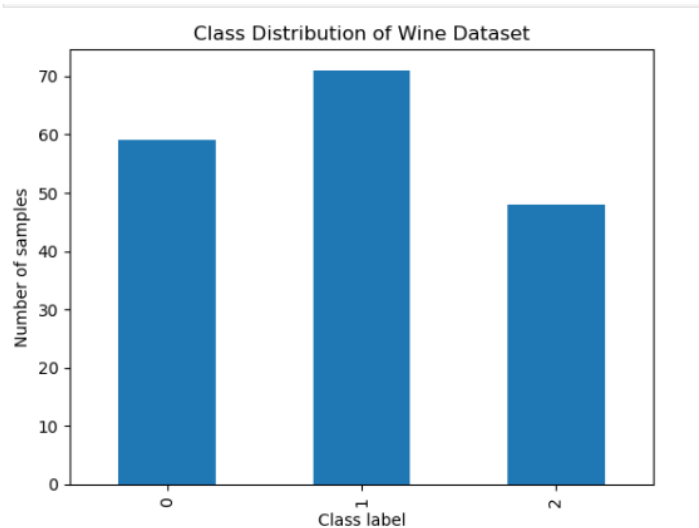
Additionally, missing value checks were carried out, and the results show that neither the target variable nor any feature have any missing values. This made sure that there was no need for data removal or imputation. Furthermore, examination of the data types revealed that the goal variable is categorical, and all predictor variables are continuous numerical features, which makes the dataset directly suitable with the supervised classification algorithms used in this project. In addition to showing the significance of preprocessing procedures like scaling for ensuring fair and stable model behaviour, these checks showed that the dataset is clean, well-structured, and appropriate for machine learning research.

4.2 Visualisations

Several plots were created to look at class balance, feature distributions, and inter-feature connections to help with preprocessing and model selection. The decisions that followed were based on evidence rather than assumption due to visual analysis.

4.2.2 Class Distribution

To look at how many samples were in each wine class, a class distribution plot was created. The results showed that, even though the three class sizes are not quite equal, their distributions are balanced and none of them are lacking.

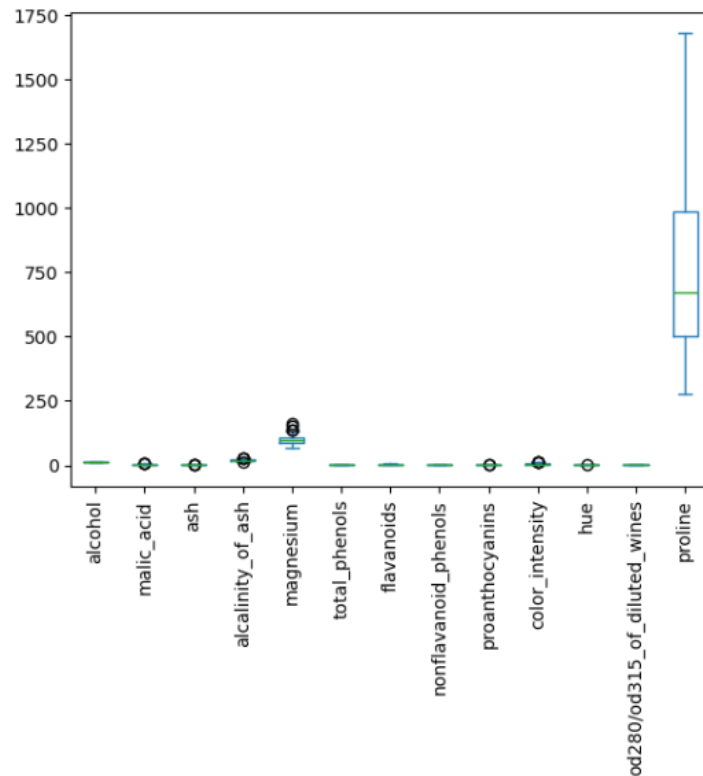


Img 5: Bar Chart for class distribution

Strong class imbalances might skew classifiers toward majority classes and require measures like resampling or class weighting, which makes this insight important. No such methods were used because the plot shows a very balanced dataset. Therefore, typical evaluation techniques based on accuracy are suitable for model comparison in this project.

4.2.3 Feature Distributions

To see the distributions, spreads, and possible outliers of all numerical features, boxplots were created. The output shows that the scale and dispersion of characteristics differ significantly. Compared to characteristics like ash or hue, traits like proline show far larger variations.

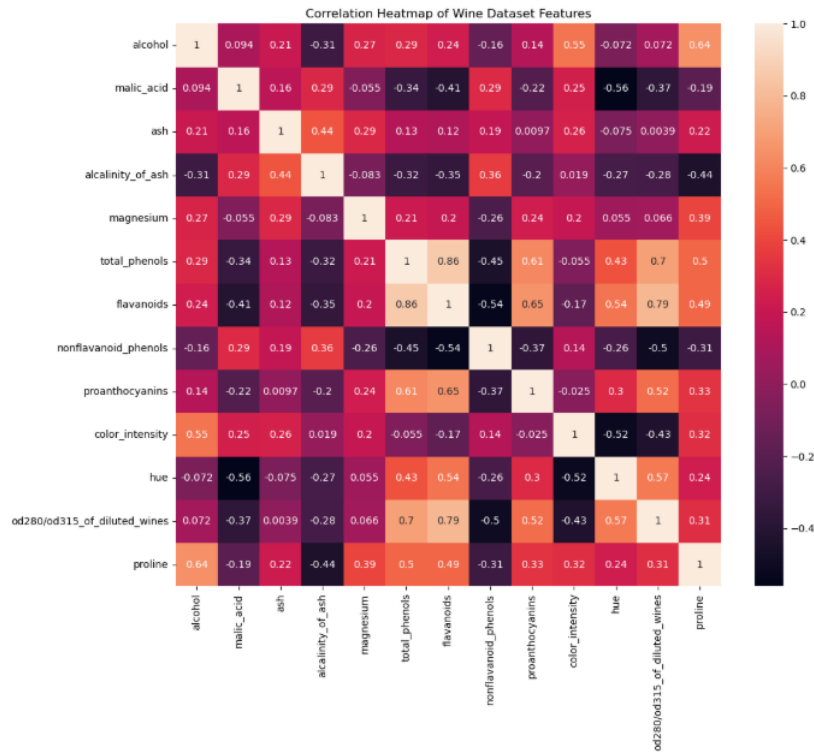


Img 6: Feature Distribution box plot

As is common with chemical measurement data, several features include skewness and the existence of outliers. Because distance-based algorithms like K-Nearest Neighbours are sensitive to both feature scale and extreme values, these observations have methodological importance. The adoption of feature normalisation before model training was directly inspired by the diversity across feature ranges. Furthermore, the boxplots showed that although outliers are present, they are not severe enough to require removal because they most likely represent actual chemical variability rather than data errors.

4.2.4 Correlation Heatmap

To investigate linear correlations between the 13 chemical characteristics, a correlation heatmap was created. The results show several positive and negative correlations, especially between phenolic-related characteristics such as total phenols, flavonoids, and od280/od315_of_diluted_wines. This shows that there is redundancy in the feature space.



Img 7: heatmap

Strong correlations can have a variety of effects on model behaviour. Correlated characteristics can have an imbalance on distance computations for distance-based classifiers, and they against independence requirements for probabilistic models. As a result, the heatmap offered convincing support for using PCA later in the process to cut down on redundancy and investigate whether a smaller set of uncorrelated components may produce comparable classification performance. Additionally, because strong feature connections could cover up the true contribution of individual qualities, the correlation study shows the significance of carefully evaluating model results.

4.3 Dataset Splitting and Feature Scaling

The dataset was split 80–20 into training and testing to assess generalisation performance. To maintain class proportions in both the training and testing sets, the split was stratified by class label. To ensure reproducibility, a fixed random state was used. This made sure that unseen data was used for model evaluation. After that, standardisation was used to apply feature scaling, changing each feature to have a zero mean and unit variance.

```
# Import the required library
from sklearn.model_selection import train_test_split

# target store in y
y = wine.target

# Train test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, stratify=y, random_state=42)

# Print the shapes for the splits
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

X_train shape: (142, 13)
X_test shape: (36, 13)
y_train shape: (142,)
y_test shape: (36,)
```

Img 8: Code snippet for train test split

```
# Import required Library
from sklearn.preprocessing import StandardScaler
import numpy as np

# Standard scaler
scaler = StandardScaler()

# Fit the scaler on the training data
X_train_scaled = scaler.fit_transform(X_train)

# Transform the test data
X_test_scaled = scaler.transform(X_test)

# Print Statements
print("Mean", np.mean(X_train_scaled, axis=0))
print("Standard Deviation", np.std(X_train_scaled, axis=0))

Mean [ 1.09781120e-15 -2.57227729e-16  9.52485352e-16  5.50127236e-16
 1.95461800e-17 -2.89283464e-16  2.00152883e-16 -9.71054223e-16
-1.57933134e-16  4.45066519e-16 -9.34307405e-16  1.03047461e-15
-2.07971355e-16]
Standard Deviation [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

Img 9: Code snippet for standardscaler

Standardisation was important for PCA, which is sensitive to feature variation, and algorithms that depend on distance estimates, such K-Nearest Neighbours. The scaler was only put on the training set before being applied to the test set to prevent data leakage.

4.4 K-Nearest Neighbours (Build from scratch)

A K-Nearest Neighbours (KNN) classifier was developed in Python to show an understanding of distance-based learning. This method made the algorithm's behaviour transparent by enabling control over neighbour selection, majority voting, and distance computation. Pairwise distances between test samples and all training samples were computed using the Euclidean distance metric. The most common class label among the k nearest neighbours was chosen as the prediction.

```

# Calculate Euclidean distances
def euclidean_distances(a, b):
    # squared norms of A (sum of squares per row)
    a_2 = np.sum(a ** 2, axis=1, keepdims=True)

    # squared norms of B (sum of squares per row), then transpose
    b_2 = np.sum(b ** 2, axis=1, keepdims=True).T

    # Use the identity
    d_2 = a_2 + b_2 - 2 * (a @ b.T)

    # distance squared should not be negative and return
    d2 = np.maximum(d_2, 0.0)
    return np.sqrt(d_2)

# KNN prediction
def knn_predict(X_train, y_train, query_x, k=5):
    # Compute distances
    distance = euclidean_distances(query_x, X_train)

    # indexes of the k smallest distances
    k_smallest = np.argsort(distance, axis=1)[: , :k]

    # neighbour labels
    neighbour = y_train[k_smallest]

    # Majority vote for each query sample
    prediction = []
    for row in neighbour:
        # times each class appears
        count = np.bincount(row, minlength=np.max(y_train) + 1)

        # class with the highest count choose
        prediction.append(np.argmax(count))

    # Return predictions
    return np.array(prediction)

```

Img 10: Code snippet for the build scratch knn model

```

# Scratch knn function to use the scikit learn tools
class scratchknn(BaseEstimator, ClassifierMixin):
    # k value
    def __init__(self, k=5):
        self.k = k

    # Store the train data
    def fit(self, X, y):
        # feature matrix
        self.X_train_ = X
        # target
        self.y_train_ = y
        # class
        self.classes_ = np.unique(self.y_train_)
        return self

    def predict(self, X):
        # Predict using build knnprediction function
        return knnprediction(self.X_train_, self.y_train_, X, k=self.k)

```

Img 11: Code snippet for build knn model to use scikit learn tools

Scikit-learn requires models to implement the fit and predict for compatibility with functions like `cross_val_score`, I developed the `scratchknn` class. I wrapped the original KNN implementation in a class called `knn_predict` that enables to efficiently use scikit-learn's tools without changing the fundamental KNN logic because it is not held to the scikit-learn API. Because distance computations immediately affect classification results, this implementation shows the KNN at prediction time and importance of feature scaling.

4.5 Other Model Training

The scikit-learn package was used to implement the Decision Tree and Gaussian Naive Bayes models in addition to the custom KNN classifier. To provide a fair comparison, these models were trained on the same pre-processed dataset. Before doing a comparative analysis, baseline performance was initially established using default hyperparameters.

```
# Import required Library
from sklearn.tree import DecisionTreeClassifier

# Decision Tree classifier, to prevent overfitting max depth set to 5
decision_treemodel = DecisionTreeClassifier(random_state=42, max_depth=5)

# Train the Decision Tree model
decision_treemodel.fit(X_train_scaled, y_train)
```

▼ DecisionTreeClassifier ⓘ ?

► Parameters

Img 12: Code snippet for decision tree model

```
# Import required Library
from sklearn.naive_bayes import GaussianNB

# Gaussian Naive Bayes model
naive_model = GaussianNB()

# Train the naive model
naive_model.fit(X_train_scaled, y_train)
```

▼ GaussianNB ⓘ ?

► Parameters

Img 13: Code snippet for Gaussian Naïve Bayes model

For these models, library implementations were chosen to ensure accuracy and computational performance, allowing to focus on model behaviour rather than implementation details.

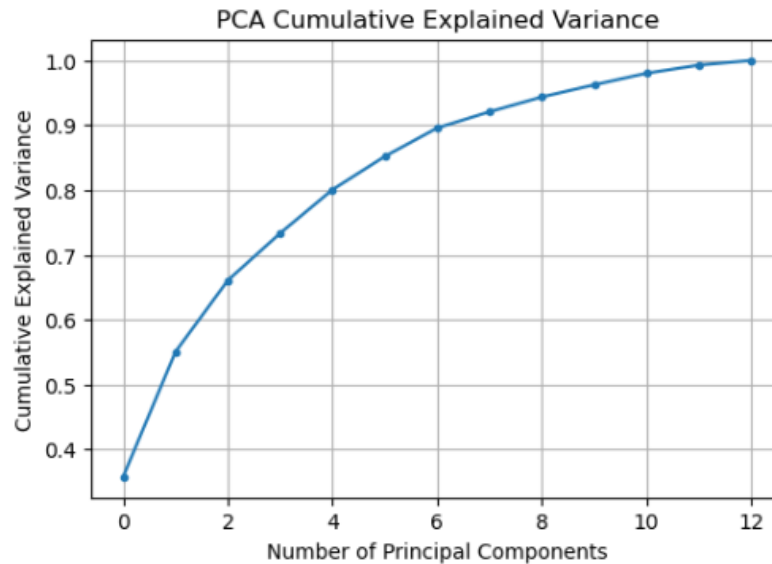
4.6 Evaluation Metrics and Cross-Validation

Classification accuracy, confusion matrices, and classification reports were used to assess the model's performance. These measures allowed for the identification of regular misclassifications by offering both general and class-level insights.

K-fold cross-validation was used to get more accurate estimations of generalisation performance due to the dataset's small size. Cross-validation was important in high-variance models like Decision Trees since it reduced dependence on a single train-test split.

4.7 Dimensionality Reduction using PCA

Following feature standardisation, PCA was used to reduce dataset redundancy and investigate the impact of dimensionality reduction on classification performance. The original associated features are converted by PCA into a new collection of components that are arranged based on how much variance they can account for.



Img 14: PCA Cumulative Explained Variance Plot

The plot shows that a small number of principal components account for a significant amount of the overall variance. The first three components got for about 70 to 75% of the variance, while the next six to seven components got for about 90%. Significant redundancy in the first feature space is shown by the fact that beyond this point, additional components only slightly improve explained variance. To produce a compact low-dimensional representation of the data for the dimensionality reduction experiment in this project, two principal components were chosen. This decision made it possible to examine the effects of significantly reducing dimensionality on the robustness and performance of the model. To reduce the impact of correlated features, increase computational efficiency, and see the trade-off between information loss and classification performance, classification models were retrained and assessed using the PCA-transformed features.

5 Results

The findings from the project's supervised classification experiments are shown in this section. Test accuracy stratified cross-validation statistics, confusion matrices, and classification reports are used to report model performance. Tables are used to summarise the results so that models can be easily compared.

5.1 Train-Test split Performance

Models were presented on the standardised training set and evaluated on the held-out test set in line with the approach in Section 4. Each model's classification accuracy is summarised in Table.

Model	Test Accuracy (2 decimal point)
KNN (Build from scratch)	0.97
Decision Tree	0.94
Gaussian Naive Bayes	0.97

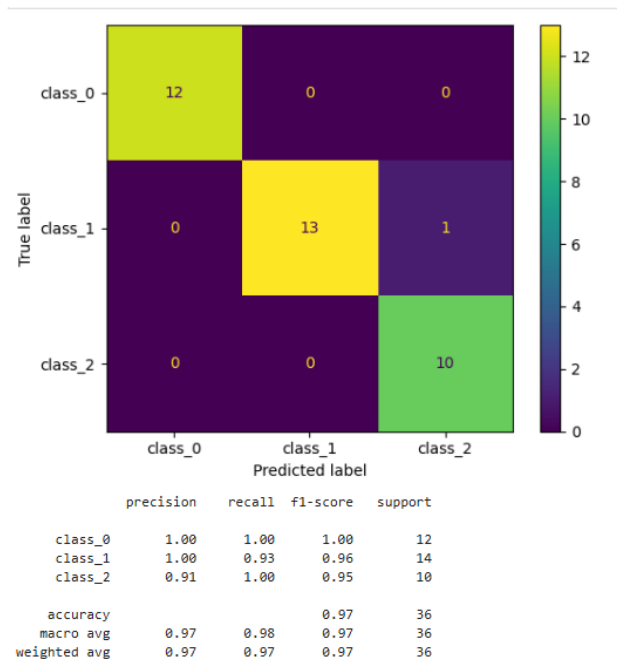
Table 1: Performance of the models after train-test split

The Gaussian Naive Bayes model and the manually built KNN classifier both obtained high test accuracy of 97%, however the Decision Tree model performed marginally worse. These findings show that while the Decision Tree is more sensitive to the training data, distance-based and probabilistic models are better adapted to the structure of the Wine dataset.

5.2 Confusion Matrix and Learning Curves

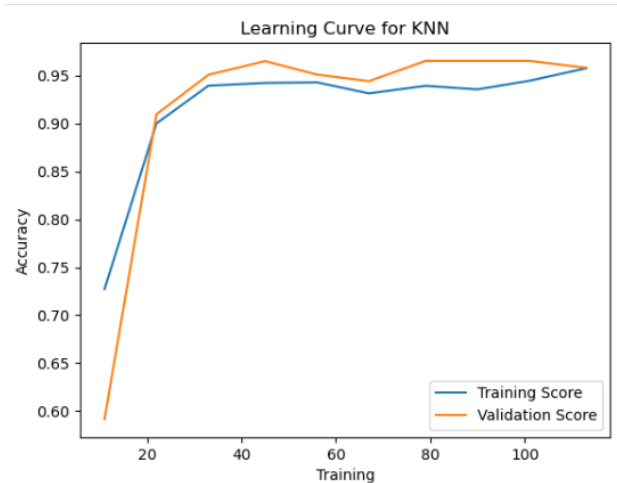
For all model, learning curves and confusion matrices were produced to give insight into class-level performance.

5.2.1 KNN



Img 15: KNN confusion matrix

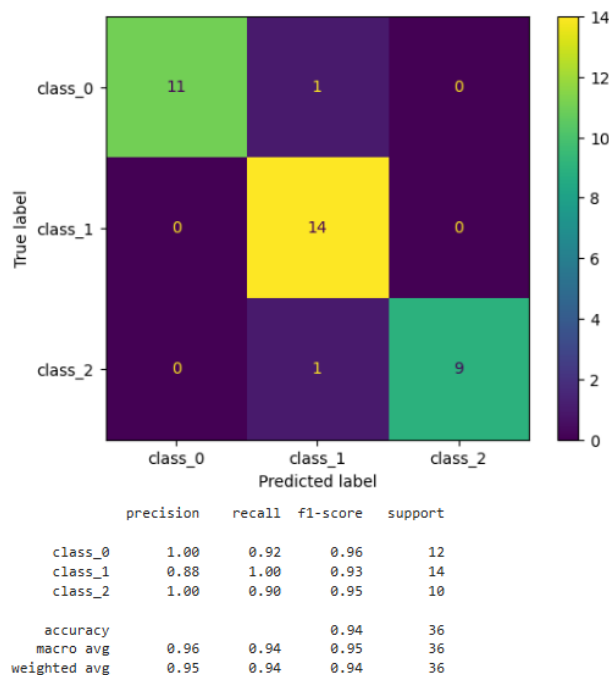
Class_0 and class_2 are perfectly classified in the confusion matrix, however class_1, which was classified to be class_2, only had one misclassification. The classification report, which has a weighted F1-score of 0.97 and strong precision and recall values, further shows balanced performance across classes. These findings show that when features are appropriately scaled, KNN is quite effective for this dataset, offering a strong baseline for comparison with other machine learning models. Overall accuracy is 97% due to consistently good precision, recall, and F1-scores in all three classes.



Img 16: Learning curve of KNN

The KNN learning curve shows that the model rapidly achieves high accuracy as the size of the training set increases, suggesting good generalisation. The model appears to be neither overfitting nor underfitting and is operating consistently across a range of training sizes, as shown by the small difference between the training and validation scores.

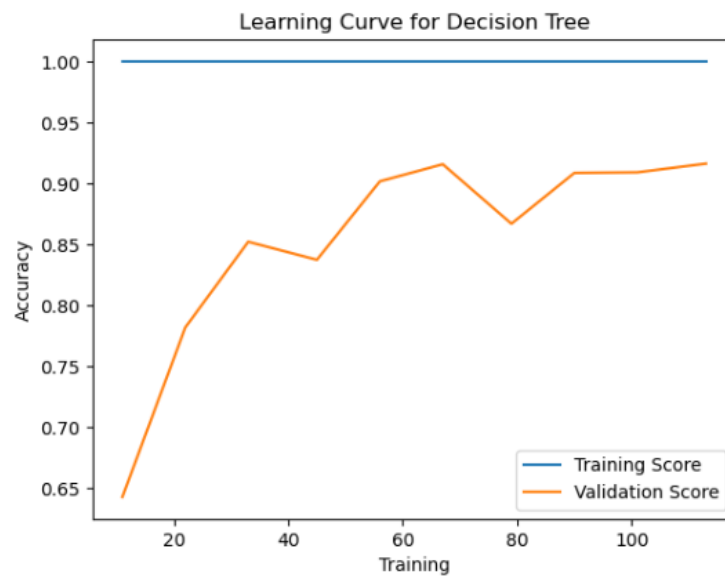
5.2.2 Decision Tree



Img 17: Confusion matrix for decision tree model

On the Wine dataset, the Decision Tree classifier obtained a test accuracy of 0.94. With only one misclassification between class_0 and class_1 and between class_2 and class_1, the confusion matrix demonstrates that most samples were correctly identified. Although overall performance is marginally worse than that of the KNN model, the classification report shows excellent

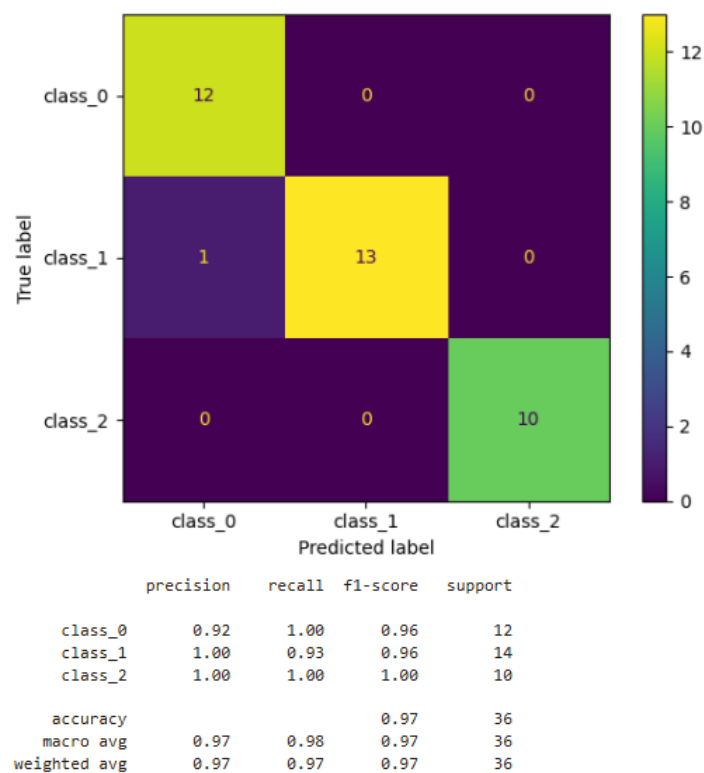
precision and recall across all classes. This tells that, in comparison to the distance-based KNN technique, the Decision Tree may be more sensitive to small variations even while it captures significant decision boundaries in the data.



Img 18: Learning Curve for Decision tree model

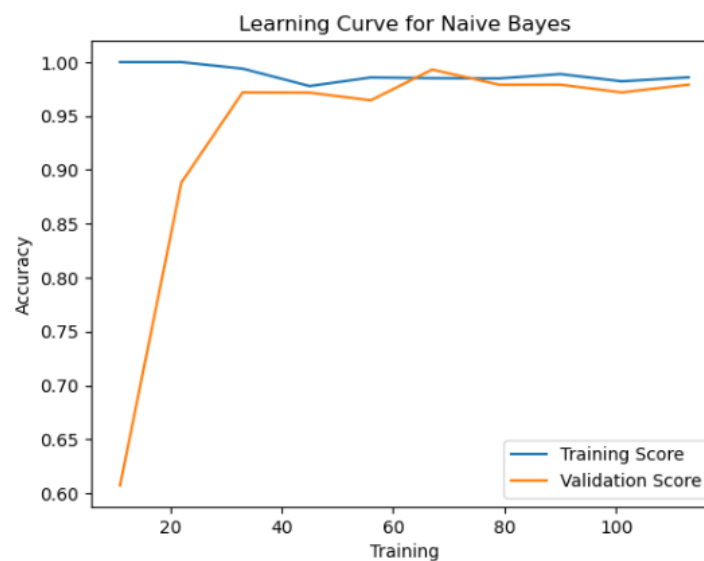
A high training score 1.0 shows that the model fits the training data well, and the Decision Tree's learning curve shows that the model significantly improves with more training data. Although there may still be some sensitivity to changes in the training data, as shown by the validation score's fluctuations, the validation score gradually rises and fluctuates, indicating that the model is learning from the data without overfitting.

5.2.3 Gaussian Naive Bayes



Img 19: Confusion matrix for Naive Bayes model

Class_2 is perfectly classified in the confusion matrix, but class_0, which was supposed to be class_1, has only one misclassification. With a weighted F1-score of 0.97, the classification report shows excellent recall and precision in all classes. The Naive Bayes model performs competitively on this dataset, showing that the class conditional feature distributions are sufficiently separable despite its simplifying independence assumption.



Img 20: Learning curve for Naïve Bayes

As the size of the training set grows, the Naive Bayes learning curve demonstrates that the model rapidly achieves high accuracy. Both the validation and training scores stabilise at high values, and they closely follow each other. This shows that there are no indications of overfitting and the model is well regularised. Even with larger training sets, the accuracy difference between training and validation is still small, telling that the model performs well when applied to new data.

Because confusion matrices show how errors are distributed throughout classes rather than being summed up by a single metric, these results highlight the significance of looking at confusion matrices in addition to accuracy.

5.3 Stratified 5-Fold Cross-Validation Results

Stratified 5-fold cross-validation was used to get a more trustworthy assessment of generalisation performance. Table below reports the average accuracy and standard deviation across folds.

Model	Mean Accuracy	Standard Deviation
Gaussian Naïve Bayes	0.97	0.0259
KNN	0.95	0.0353
Decision Tree	0.90	0.0586

Table 2: Mean Accuracy and Standard Deviation across Folds

Gaussian Naive Bayes offers the best and most consistent performance across folds with a low standard deviation, according to the cross-validation findings. The Decision Tree model's sensitivity to training data divisions is further demonstrated by its poor performance with increased variability. Also, the KNN model shows significant cross-validation performance, with a high mean accuracy with comparatively low variability, indicating strong generalisation when assessed over multiple folds. Because single train–test split results can overestimate performance, these findings highlight the significance of cross-validation when assessing models on datasets.

5.4 Dimensionality Reduction using PCA results

Two principal components of PCA-transformed features were used to retrain the models. Classification accuracy is compared before and after dimensionality reduction in Table below.

Model	Test Accuracy	PCA (2 components)
KNN	0.97	0.92
Decision Tree	0.94	0.89
Gaussian Naïve Bayes	0.97	0.94

Table 3: PCA Results table

Severe dimensionality reduction results in a fall in classification accuracy for all models, but performance is still competitive. The least amount of accuracy loss is shown with Gaussian Naive Bayes, showing its adaptability to information loss. These findings support the idea that PCA can reduce dimensionality while retaining a

portion of the ability to predict and that the original feature space has significant redundancy.

Overall, the findings show that, with the right preprocessing and validation techniques, more straightforward models like KNN and Gaussian Naive Bayes can perform well on the Wine dataset. While PCA verified the existence of redundancy in the original feature space, cross-validation shows significant variations in model stability that are not seen from a single train-test split.

6 Evaluation

To investigate the effects of preprocessing, validation techniques, and dimensionality reduction on classification performance, this study compared and seen several supervised classification algorithms on a typical wine dataset. The assessment that follows critically shows how successfully goal was achieved, the experiment advantages, and the dataset and decisions limits.

6.1 Model Performance Relative to Aim

The findings show that rather than only increasing predicted accuracy, the project's main goal of comparing algorithmic behaviour was successfully accomplished. The analysis showed that stability should be checked by repeated validation rather than depending on a single split, even though several models KNN and Gaussian Naive Bayes achieved excellent test accuracy. Significant variations in model stability were revealed by stratified cross-validation, with Decision Trees displaying greater variability and Gaussian Naive Bayes continuously demonstrating strong performance and low variance.

The reliability of some models would have been overestimated if a single train-test split had been used, so this is important. As a result, the evaluation highlights the need of validation techniques for analysing classification performance, especially for small datasets with significant variable effects.

6.2 Advantage of the Approach

This project's approach is one of its main advantages. Preprocessing choices, such dimensionality reduction and feature standardisation, were made based on facts rather than guess due to the use of exploratory data analysis. The project got even stronger by the addition of a personally implemented KNN classifier, which demonstrated algorithmic understanding beyond the use of library methods.

Using a stratified technique, cross-validation was implemented correctly, maintaining class proportions across folds. This decision improved the evaluation's dependability and fit in nicely with the small size of the dataset. The trade-off between performance and generalisation stability was better understood by comparing the outcomes of cross-validation and single split evaluation.

Furthermore, by specifically examining the effect of information loss on classification accuracy, the PCA experiment improves the analysis. The conclusion that there is

significant redundancy in the dataset is supported by the fact that performance remained competitive even after extensive dimensionality reduction.

6.3 Impact of the Limitation of dataset

The Wine dataset has several restrictions that limit how broadly results can be applied, even if it is appropriate for comparative purposes. Particularly for high-variance models like Decision Trees, the comparatively small sample size increases the impact of data partitioning. The greater variability seen during cross-validation was evidence of these limitations.

Additionally, the dataset is linearly separable and well-structured, which encourages simpler models and reduces the need for complex decision limits. Because of this, the excellent results of Gaussian Naive Bayes and KNN could not translate to real-world datasets that are noisier or have more features. This highlights that high accuracy on benchmark datasets should be viewed carefully and limits the findings outside validity.

6.4 Evaluation of algorithms

The evaluation of the custom-built KNN model on the test set confirmed the implementation's accuracy. The custom KNN model's cross-validation performance was reliable and consistent, highlighting the significance of taking bias and variance into account when selecting a classifier. This highlights the difficulties with distance-based methods, especially how vulnerable they are to local neighbourhood structure and data partitioning, indicating the importance of thorough model validation to ensure reliable results.

Gaussian Naive Bayes showed outstanding durability despite its simple feature independence assumptions. This result highlights an important in machine learning practice where models can perform well in practice even if they do not perfectly follow to theoretical assumption, especially if the dataset structure supports the model's bias.

Although decision trees had advantages in terms of interpretability, their instability confirmed that they were prone to overfitting on short datasets. This analysis backs the choice to go deeper tree tuning because such optimisation would probably result in even more variance without appreciable improvements in generalisation.

6.5 Methodological Constrains and Future Improvements

Several methodological improvements could be in further work, even if the project was effective within its specified scope. Although hyperparameter optimisation such as adjusting the value of k in KNN, it was purposefully restricted here to keep the focus on comparative comprehension rather than optimisation. In the future, exploring feature selection strategies or using advanced dimensionality reduction techniques like t-SNE may improve model performance by addressing high-dimensionality problems. Furthermore, using techniques like XGBoost or ensemble

approaches like Random Forest could increase classification robustness and accuracy.

Further understanding of the bias–variance trade-off may be obtained by assessing ensemble approaches or regularised linear classifiers. The outside validity of the findings would be enhanced and the robustness of simpler models under more difficult circumstances would be tested by applying the same methodology to larger or noisier datasets.

The project highlights the significance of validation technique, model stability, and interpretability rather than viewing high accuracy as the only measure of success. The results underline that performance indicators need to be situated within dataset features and experimental design, but they also demonstrate that simpler models can perform competitively when data is well-prepared. The research is successful in delivering a fair and careful comparison of supervised classification algorithms in connection to the stated goal, providing insights that go beyond numerical findings to useful model selection.

7 Conclusions

To investigate the effects of preprocessing, validation techniques, and dimensionality reduction on classification performance, this study compared and investigated several supervised classification algorithms on a wine dataset. The findings shows that more straightforward models like Gaussian Naive Bayes and K-Nearest Neighbours can perform well when proper preprocessing is used. Specifically, despite its simplifying independence assumptions, Gaussian Naive Bayes consistently showed high accuracy and stability across test-set evaluation and stratified cross-validation, supporting findings in the literature that robust performance does not require strict rules to theoretical assumptions (Jiang, 2024).

The review also highlights the significance of dimensionality reduction and validation technique in applied machine learning. Particularly for higher-variance models like Decision Trees, cross-validation showed significant variations in model stability that were not seen from a single train–test split (Khilari et al., 2022). Furthermore, as competitive classification performance was maintained even after rapid dimensionality reduction, Principal Component Analysis verified significant redundancy in the original feature space, which is consistent with earlier studies on wine classification datasets (Pradhana, Batubulan, and Kotama, 2024). Overall, the project achieves its goal by showing that increasing predicted accuracy is just as important as understanding algorithm behaviour, validation robustness, and data properties.

8 References

scikit-learn (2024) Wine recognition dataset. (Online) Available at: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_wine.html (Accessed on : 22nd December 2025).

Jiang, L. (2024) *Research on Wine Classification Based on BP Neural Network*. (Online) Available at: https://www.researchgate.net/publication/383375490_Research_on_Wine_Classification_Based_on_BP_Neural_Network (Accessed on: 22nd December 2025).

Khilari, P., Patil, S., Pawar, A. and Jadhav, R. (2022) *Analysis of Machine Learning Algorithms to Predict Wine Quality*. (Online) Available at: https://www.researchgate.net/publication/360456284_Analysis_of_Machine_Learning_Algorithm_to_predict_Wine_Quality

(Accessed on: 22nd December 2025).

Pradhana, A.A., Batubulan, K.S. and Kotama, I.W. (2024) *Analysis of K-Nearest Neighbour Algorithm for Wine Quality Classification*. (Online) Available at: <https://infoteks.org/journals/index.php/jsikti/article/view/231>

(Accessed on: 22nd December 2025).

Cao, Y., Chen, H. and Lin, B. (2022) *Wine Type Classification Using Random Forest Model*. (Online) Available at:

https://www.researchgate.net/publication/362436263_Wine_Type_Classification_Using_Random_Forest_Model (Accessed on: 22nd December 2025).

Abhinav (2025) 'Gini Impurity and Entropy in Decision Tree', (Online) Available at:

<https://www.geeksforgeeks.org/machine-learning/gini-impurity-and-entropy-in-decision-tree-ml/> (Accessed: 25th December 2025).

Neha (2025) 'Bayes' Theorem', (Online). Available at:

<https://www.geeksforgeeks.org/maths/bayes-theorem/> (Accessed: 25th December 2025).

Baliyan, M. (2020) 'Implementation of K Nearest Neighbors from Scratch using Python' (Online). Available at: <https://www.geeksforgeeks.org/machine-learning/implementation-of-k-nearest-neighbors-from-scratch-using-python/>

(Accessed: 25th December 2025).